Fine-Tuning LLMs from Zero

A plain-language study guide: tokenizers, embeddings, BERT, Hugging Face, LoRA & QLoRA

Revision notes for students · October 2026

Includes cheat sheets, little-known details,
and a small program that ties everything together

Contents

1. The big picture: what an LLM really is
2. What fine-tuning is, and why we need it
3. Hugging Face: the Hub and the libraries
4. Tokenizers: cutting text into pieces
5. Where the ID numbers come from
6. Special tokens: [CLS], [SEP], [PAD], [UNK], [MASK]
7. Padding and truncation
8. attention_mask and token_type_ids
9. The three tokenizer algorithms: BPE, WordPiece, Unigram/SentencePiece
0. Embeddings: turning IDs into meaning
1. The Transformer: how words look at each other
2. Model families: BERT vs GPT vs T5
3. AutoModel, AutoTokenizer and task heads
4. Memory and precision: why big models are hard to train
5. PEFT, LoRA and QLoRA
6. The Python libraries in depth
7. Master cheat sheet
8. Self-check questions
9. The small program that ties it all together

> **How to use this guide**
> Read sections 1–3 for the big picture, 4–13 for how text becomes predictions, and 14–16 for how fine-tuning is done cheaply. Then run the program in section 19 and match each printed line back to the section that explains it.

1. The big picture: what an LLM really is

An LLM (Large Language Model) is a very large mathematical function. Text goes in, numbers come out, and those numbers are turned back into text or into a decision.

A model **cannot read letters**. Computers only understand numbers. So every model runs the same four-step pipeline:

```
"I love pizza"
      |
      v  (1) TOKENIZER    cuts text into pieces, gives each piece an ID number
["i", "love", "pizza"]  ->  [1045, 2293, 10733]
      |
      v  (2) EMBEDDING LAYER   turns each ID into a list of 768 numbers ("meaning vector")
[[0.12, -0.4, ...], [0.88, 0.1, ...], [...]]
      |
      v  (3) TRANSFORMER LAYERS   words "look at" each other and update their meaning
contextual vectors (each word now knows its neighbours)
      |
      v  (4) HEAD   a small final layer for your specific task
"positive review: 97%"   or   "next word: ' tonight'"
```

Remember this picture

Tokenizer → Embedding → Transformer layers → Head. Every topic in this guide is a closer look at one of these four boxes.

2. What fine-tuning is, and why we need it

Kitchen analogy

Pre-training is culinary school. The model reads a huge part of the internet and learns language in general: grammar, facts, how sentences flow. It costs millions of dollars and huge GPU clusters. **You never do this yourself.**

Fine-tuning is a trained chef joining a specific restaurant. They already know how to cook; you only teach them *your* menu, style and plating. It is cheap, fast, and needs a small amount of data (hundreds to a few thousand examples).

Why fine-tune instead of just prompting a big chatbot?

Reason	Example
A specific task	Classify customer complaints into 5 categories
A specific style or format	Always reply in your brand voice; always output JSON
Domain language	Food-safety compliance wording, legal or medical terms
Cost and speed	A small fine-tuned model can beat a giant model on one narrow task and runs cheaply
Privacy	Run it on your own machine so the data never leaves

What few people say out loud

Fine-tuning is mainly good at teaching **behaviour, format and style**. It is not the best way to teach **new facts**. If you want a model to "know" your documents, the usual tool is **RAG** (Retrieval-Augmented Generation): search your documents, then paste the relevant parts into the prompt. Many beginners fine-tune when they should have used RAG.

The three ways to fine-tune

1. **Full fine-tuning**: update *every* weight in the model. Best quality, but heavy. Fine for small models like BERT.
2. **PEFT / LoRA**: freeze the original model and train tiny "adapter" layers (often under 1% of the parameters). Almost as good, and fits on cheap GPUs. This is how most people fine-tune LLMs today.
3. **QLoRA**: LoRA on top of a model compressed to 4-bit. Lets you fine-tune a 7–8 billion parameter model on a free Colab GPU.

Section 15 explains LoRA and QLoRA in full.

---

3. Hugging Face: the Hub and the libraries

Think of Hugging Face as **GitHub plus an app store for AI models**. It has two halves.

A) The Hub (huggingface.co)

- **Models**: hundreds of thousands of pre-trained models (BERT, Llama, Mistral, Qwen, Gemma…) you can download for free.
- **Datasets**: ready-made training data such as movie reviews, Q&A pairs and translations.
- **Spaces**: small demo apps people host.
- Every model has a **model card** (its README). Always read it: it gives the license, intended use and prompt format.

Models are named `owner/model-name`, for example `google-bert/bert-base-uncased` or `meta-llama/Llama-3.1-8B`. Some models are **gated**: you must accept their license on the website and log in with an access token before downloading.

> **Tiny detail**
> Downloaded models are cached in `~/.cache/huggingface/`. They can be many gigabytes, so this is the first folder to check when your disk fills up.

B) The Python libraries

Library	Job in one line
`transformers`	Load and run models and tokenizers. The core library.
`datasets`	Load and process training data efficiently
`tokenizers`	Very fast tokenizer engine written in Rust (used automatically)
`peft`	LoRA and other adapter methods
`trl`	Training chat / instruction models (SFTTrainer, DPO)
`accelerate`	Runs the same code on CPU, one GPU or many GPUs
`bitsandbytes`	4-bit and 8-bit compression (needed for QLoRA)
`evaluate`	Metrics like accuracy and F1
`torch` (PyTorch)	Not Hugging Face, but the maths engine underneath everything

Section 16 covers each library in depth.

4. Tokenizers: cutting text into pieces

Why not just split on spaces?

Approach	Problem
Whole words	Millions of words exist; typos, names and new words become "unknown"
Single characters	Tiny vocabulary, but sentences get very long and each letter carries little meaning
Subwords (the winner)	Common words stay whole; rare words split into meaningful known pieces

```
"tokenization"  ->  ["token", "##ization"]
"unbelievable"  ->  ["un", "##believ", "##able"]     (illustrative BERT-style split)
```

What the tokenizer returns

For `"I love pizza"` with BERT you get a dictionary of three lists:

```
{
  'input_ids':      [101, 1045, 2293, 10733, 102],
  'token_type_ids': [0,   0,    0,    0,     0],
  'attention_mask': [1,   1,    1,    1,     1]
}
```

Key	Question it answers	Values
`input_ids`	Which tokens are these?	Line numbers in the vocabulary
`attention_mask`	Which tokens are real?	1 = real, 0 = padding
`token_type_ids`	Which sentence is each token from?	0 = first, 1 = second

Sections 5–8 explain each of these in detail. Exact ID numbers can vary slightly by vocabulary version; run the program in section 19 to see the real ones.

Tokenizer details most tutorials skip

1. **The tokenizer must always match the model.** BERT's tokenizer with a Llama model gives garbage, because the same ID means a different word in each vocabulary.
2. **"uncased" means text is lowercased first.** In `bert-base-uncased`, "Apple" and "apple" are identical. "cased" models keep capitals, which matters for names.
3. **Maximum length.** BERT can see only **512 tokens**. Longer text must be truncated or split. Modern LLMs handle thousands to hundreds of thousands.
4. **Rule of thumb:** in English, 1 token ≈ ¾ of a word ≈ 4 characters. Hindi, Gujarati and other non-Latin scripts often need **2–4× more tokens** for the same meaning on English-centric models.
5. **Numbers tokenize strangely.** "2026" may be one token or "20" + "26". This is one reason LLMs struggle with arithmetic.
6. **"Fast" tokenizers** (Rust-based, the default) can tell you which characters each token came from (`offset_mapping`), useful for highlighting names in text.

5. Where the ID numbers come from

The short answer

An ID is simply a **line number in the vocabulary file**. For BERT the file is a plain text file called `vocab.txt`, one token per line. Line 0 is ID 0, line 1 is ID 1, and so on.

```
Line 0      [PAD]
Line 1      [unused0]
 ...
Line 99     [unused98]
Line 100    [UNK]
Line 101    [CLS]
Line 102    [SEP]
Line 103    [MASK]
Line 104    [unused99]
 ...        (single characters, punctuation, digits)
Line 1045   i
 ...
Line 2293   love
 ...
Line 30521  (last token)
```

When the tokenizer sees "love", it finds the line where "love" sits (2293), and that is the ID. **The number has no meaning by itself.** It is an address, like a house number on a street.

Who decided "love" goes on line 2293?

The vocabulary is built **once, before the model is trained**, by running a tokenizer-training algorithm over a huge pile of text (for BERT: Wikipedia plus a large book collection). Three stages:

1. **Reserve special tokens** at fixed positions ([PAD], [UNK], [CLS]...).
2. **Add the basic characters** (letters, digits, symbols), so nothing is ever completely unknown.
3. **Add learned word-pieces** one by one (section 9 explains how) until the target size is reached. For BERT that was 30,522.

In BPE tokenizers (GPT, Llama) order carries meaning too: pieces learned earlier, which are more common, usually have **smaller IDs**.

Details few people know

- **The `[unused]` tokens are empty slots** Google left on purpose. You can rename one to your own word (e.g. `[MYBRAND]`) without changing the vocabulary size or resizing the model.
- **Once a model is trained, its vocabulary is frozen.** The embedding table has exactly one row per ID. Change the vocabulary and rows no longer match words.
- **Adding tokens** needs two steps: `tokenizer.add_tokens([...])` then `model.resize_token_embeddings(len(tokenizer))`. The new rows start random and must be learned during fine-tuning.

Model	Vocabulary size
BERT	30,522
T5	32,000
Llama 2	32,000
GPT-2	50,257
Llama 3	128,256

A bigger vocabulary means fewer tokens per sentence (faster, cheaper, better for non-English text) but a bigger embedding table.

6. Special tokens: [CLS], [SEP], [PAD], [UNK], [MASK]

Special tokens are **stage directions** for the model. They are not real words; they are signals, like punctuation for the model's world.

ID	Token	Meaning
101	`[CLS]`	"Start here. My seat collects the summary of the whole sentence."
1045	`i`	Real word
2293	`love`	Real word
10733	`pizza`	Real word
102	`[SEP]`	"This sentence ends here."

[CLS] (classification), ID 101

> **Meeting analogy**
> One extra person sits at the head of the table. They say nothing of their own, but listen to everyone, and at the end they summarise the whole discussion. That is [CLS].

Through self-attention, the [CLS] vector "listens" to every other word across all 12 layers. By the end, its 768 numbers form a **summary of the whole sentence**. A classification head (e.g. positive/negative review) reads **only the [CLS] vector**.

Why it works: during pre-training BERT played **Next Sentence Prediction**: given two sentences, guess whether the second really follows the first. The answer had to come from [CLS], so the model learned to pack sentence-level meaning into it.

Detail: GPT-style models have no [CLS]. They read left to right, so the **last token** is the one that has seen everything; classifiers built on them use the last token instead.

[SEP] (separator), ID 102

A full stop between sentences. With two texts it sits between them and at the end:

```
[CLS] where is the lido [SEP] the lido is in hyde park [SEP]
```

[PAD] (padding), ID 0

Empty filler so all sentences in a batch have equal length. See section 7.

[UNK] (unknown), ID 100

"I have never seen this." Rare with subword tokenizers because almost anything can be built from smaller pieces; it still appears with some emojis or unusual scripts in BERT. Byte-level BPE (GPT-style) **never** produces it.

[MASK], ID 103

A blank to fill in. In pre-training, 15% of words were chosen and BERT had to guess them:

```
"I love [MASK] with olives"  ->  BERT guesses "pizza"
```

This is how BERT learned language without humans labelling data. **Detail:** of the chosen 15%, only 80% were replaced by [MASK], 10% by a random word, and 10% left unchanged, so BERT could not rely on seeing [MASK], which never appears in real text.

Special tokens in modern chat LLMs

Llama, Qwen and similar models use tokens such as `<|begin_of_text|>`, `<|eot_id|>` (end of turn) and `<|start_header_id|>user<|end_header_id|>` to mark who is speaking. Getting these wrong is the **number one reason a fine-tuned chatbot rambles forever or talks to itself**. The fix is the tokenizer's built-in **chat template**.

7. Padding and truncation

The problem

GPUs are fastest when they process many sentences at once (a **batch**). A batch must be a perfect **rectangle**, like an egg tray: every row needs the same number of slots. But sentences have different lengths.

The fix: padding

```
"I love pizza"                -> [101, 1045,  2293, 10733,  102,     0,   0]
"Pizza is great with olives"  -> [101, 10733, 2003,  2307, 2007, 26066, 102]
"Yum"                         -> [101, 22088,  102,     0,    0,     0,   0]
```

> **Lunchbox analogy**
> Every lunchbox must be the same size to stack in the van, so you stuff tissue paper into the half-empty ones. The tissue paper is not food; it only fills space. [PAD] is the tissue paper.

Truncation, padding's opposite

If a sentence is **too long** (over 512 tokens for BERT), the end is cut off. That is **truncation**.

Padding settings

Setting	What it does	When to use
`padding="max_length"`	Pads everything to a fixed length, e.g. 512	Simple but wasteful
`padding=True` / `"longest"`	Pads to the longest sentence in this batch	Better
Dynamic padding with `DataCollatorWithPadding`	Pads each training batch separately	**Best**; much faster

> **Common beginner mistake**
> Padding everything to 512 when most sentences are 20 tokens means roughly **25× more work than needed**. Many slow training runs are slow for exactly this reason.

- **Padding side:** BERT pads on the right. Decoder models (GPT, Llama) must pad on the **left when generating**, otherwise the model continues from a pile of padding and outputs nonsense.
- **Missing pad token:** many LLMs have none, so you will often see `tokenizer.pad_token = tokenizer.eos_token` — "borrow the end-of-text token as filler."

8. attention_mask and token_type_ids

attention_mask: "which tokens are real?"

1 = real, pay attention. 0 = padding, ignore it.

```
tokens:          [CLS]   yum  [SEP]  [PAD] [PAD] [PAD] [PAD]
input_ids:       [101, 22088,  102,     0,    0,    0,    0]
attention_mask:  [  1,     1,    1,     0,    0,    0,    0]
```

Why it is needed: in self-attention every token looks at every other token. Without the mask, "yum" would also look at four [PAD] tokens and its meaning would be polluted by empty filler. With the mask, attention to padding is set to zero.

> **Highlighter analogy**
> You are reading a form. Highlighted boxes contain answers; grey boxes are blank and you skip them. The attention mask is the highlighter.

> **Silent bug**
> If you forget to pass `attention_mask`, the model often still runs, just with quietly worse results. Always pass the whole tokenizer output: `model(**inputs)`. The `**` unpacks all keys at once.

token_type_ids: "which sentence does this token belong to?"

0 = sentence A, 1 = sentence B. It matters only when BERT receives **two texts at once**, like a question plus a passage:

```
tokenizer("Where is the Lido?", "The Lido is in Hyde Park.")

tokens:         [CLS] where is the lido ? [SEP] the lido is in hyde park . [SEP]
token_type_ids: [  0,   0,   0,  0,  0,  0,  0,   1,   1,  1, 1,  1,   1,  1,  1]
attention_mask: [  1,   1,   1,  1,  1,  1,  1,   1,   1,  1, 1,  1,   1,  1,  1]
```

> **Shirt-colour analogy**
> Two people in a conversation wear different coloured shirts. All the words are in one line, but the colour tells the model who said what.

With one sentence it is all zeros, which is why "I love pizza" showed `[0, 0, 0, 0, 0]`. Internally BERT turns these 0s and 1s into the **segment embedding** (section 10).

Detail: RoBERTa, DistilBERT, GPT, Llama and most modern models do **not** use token_type_ids, so their tokenizers do not return them. Do not be confused when the key is missing.

9. The three tokenizer algorithms

All three answer one question: **which subword pieces go into the vocabulary, and how do we split new text with them?**

A) BPE (Byte-Pair Encoding)

History

- **1994**: Philip Gage invented BPE as a **file-compression** method, nothing to do with AI. Find the most common pair of bytes, replace it with one new symbol, repeat.
- **2015–2016**: Rico Sennrich, Barry Haddow and Alexandra Birch (University of Edinburgh) applied the same trick to the rare-word problem in machine translation. Their paper made BPE the NLP standard.
- **2019**: OpenAI's **GPT-2** introduced **byte-level BPE**, starting from the 256 possible bytes instead of letters. GPT-2's 50,257 tokens = 256 bytes + 50,000 learned merges + 1 end-of-text token.
- **Today**: GPT-3/4, Llama 3, Qwen, Mistral and most modern LLMs use BPE. OpenAI's fast implementation is called `tiktoken`.

How it learns, step by step

Tiny training text with word counts: `cat ×5, cats ×3, hat ×4, that ×2`

```
Step 0  Split into characters. Vocabulary: c a t s h
        c a t (x5)   c a t s (x3)   h a t (x4)   t h a t (x2)

Step 1  Count neighbouring pairs:
        a+t = 5+3+4+2 = 14   <- most frequent
        c+a = 8   h+a = 6   t+s = 3   t+h = 2
        Merge "a"+"t" -> "at"          MERGE RULE #1: a+t -> at

Step 2  c at (x5)  c at s (x3)  h at (x4)  t h at (x2)
        c+at = 8  <- winner   h+at = 6   at+s = 3   t+h = 2
        Merge -> "cat"                 MERGE RULE #2: c+at -> cat

Step 3  h+at = 6 -> "hat"               MERGE RULE #3: h+at -> hat

... continue until the vocabulary reaches its target size
```

How it splits a word it has never seen: "chats"

```
start:     c  h  a  t  s
rule #1:   c  h  at  s       (a+t -> at)
rule #2:   no "c"+"at" pair here, skip
rule #3:   c  hat  s         (h+at -> hat)
result:    ["c", "hat", "s"]
```

The rules are replayed **in the order they were learned**.

Why "byte-level" matters

Every character (emoji, Gujarati, Chinese, typos) is made of bytes, and all 256 bytes are in the vocabulary. So **byte-level BPE never produces [UNK]**; at worst a rare character becomes 2–4 byte tokens. This is also why Devanagari and Gujarati cost more tokens: each letter is 3 bytes in UTF-8.

The `Ġ` symbol

GPT-2 attaches a space to the **start** of the next word, so `" pizza"` and `"pizza"` are **different tokens**. It displays that space as `Ġ` (e.g. `Ġpizza`) purely for readability. This is why "Hello" and " Hello" get different IDs.

B) WordPiece

History

- **2012**: Mike Schuster and Kaisuke Nakajima at Google created it for **Japanese and Korean voice search**, languages with many characters that do not always use spaces.
- **2016**: used in Google Neural Machine Translation (Google Translate).
- **2018**: Google's **BERT** used it and made it famous. DistilBERT and ELECTRA use it too.
- **Little-known fact**: Google never fully published the original training code. Hugging Face and others implement a reconstruction from the papers.

How it differs from BPE

BPE merges the pair that appears **most often**. WordPiece merges the pair whose parts **belong together most strongly**:

```
score(A, B) =       how often A and B appear together
              ------------------------------------------
              (how often A appears) x (how often B appears)
```

Example: `q` and `u` are not very common overall, but whenever `q` appears it is almost always followed by `u`. WordPiece sees they are inseparable and merges them early. `e` + `s` may appear together thousands of times, but each also appears everywhere separately, so their score is lower.

> BPE counts **popularity**. WordPiece measures **how strongly two pieces belong together**.

How it splits text: greedy longest-match-first

```
"unbelievable"
  longest vocabulary match at the start  -> "un"
  remaining "believable", longest match   -> "##believ"
  remaining "able"                        -> "##able"
  result: ["un", "##believ", "##able"]
```

The `##` symbol means "I am attached to the previous piece; no space before me." So `play` (start of word) and `##play` (inside a word) are **two different tokens** with different IDs.

Detail: before WordPiece runs, BERT does **pre-tokenization**: split on spaces and punctuation, and for uncased models lowercase and strip accents. So "Café!" becomes `["cafe", "!"]`.

C) Unigram and SentencePiece

> **People mix these up constantly**
> **SentencePiece is a tool (library). Unigram is an algorithm.** SentencePiece can run either Unigram or BPE.

History

- **2018**: Taku Kudo (Google) published the **Unigram** language-model tokenizer in a paper on "subword regularization".
- **2018**: Kudo and John Richardson released **SentencePiece** as open source.
- **Used by**: T5 (Unigram), ALBERT, XLNet, mBART; Llama 1 and 2 used SentencePiece running BPE (32,000 tokens). Llama 3 switched to a tiktoken-style BPE with ~128K tokens.

How Unigram learns: the opposite direction

BPE and WordPiece **build up** from characters. Unigram **cuts down**: start with a huge vocabulary of possible substrings, then repeatedly remove the pieces whose removal hurts least, until the target size is reached.

> **Sculpting analogy**
> BPE adds clay. Unigram carves away marble.

How it splits text: the most probable split

Each vocabulary token has a probability. Unigram considers the possible splits and picks the most probable one overall:

```
"hugs" could be:
  ["hug", "s"]         P(hug) x P(s)           = high   <- chosen
  ["h", "ugs"]         P(h) x P(ugs)           = low
  ["h","u","g","s"]    product of four         = much lower
```

Its special trick, subword regularization: during training you can randomly sample different splits of the same word (`["hug","s"]` one time, `["hu","gs"]` another). This makes models more robust to typos. BPE and WordPiece are deterministic: one word, one split.

What SentencePiece adds

1. **Spaces are normal characters.** Each space becomes `▁` (U+2581), so "I love pizza" → `["▁I", "▁love", "▁pizza"]`.
2. **No pre-tokenization needed.** It does not assume spaces separate words, so it works naturally for Japanese, Chinese and Thai.
3. **Perfectly reversible.** You can rebuild the exact original text, spaces included. WordPiece cannot: after lowercasing and accent-stripping, "Café" is gone.

The three side by side

	BPE	WordPiece	Unigram (SentencePiece)
Invented	1994 (compression), 2015 (NLP)	2012, Google	2018, Google
Learns by	Adding the most frequent pair	Adding the most strongly linked pair	Removing the least useful pieces
Splits by	Replaying merge rules in order	Greedy longest match	Most probable overall split
Space marker	`Ġ` (space before word)	`##` (continues a word)	`▁` (space before word)
Can produce [UNK]?	Never (byte-level)	Yes, rarely	Rarely (byte fallback)
Used by	GPT-2/3/4, Llama 3, Qwen, Mistral	BERT, DistilBERT	T5, ALBERT, Llama 1/2 (BPE mode)

Practical takeaway

You will almost never train a tokenizer yourself when fine-tuning; you use the one that came with the model. Knowing how they work explains the odd behaviour you will see: `##`, `Ġ`, `▁`, strange number splits, and why Gujarati costs more tokens.

10. Embeddings: turning IDs into meaning

An ID like 10733 ("pizza") is just a label. 10733 is not "bigger" than 2293 in any useful way. So the model keeps a large lookup table called the **embedding matrix**:

```
bert-base vocabulary = 30,522 tokens
each token  ->  a vector of 768 numbers
matrix shape = 30,522 rows x 768 columns

Token 10733 means: "grab row 10733". That row is the token's embedding.
```

Why lists of numbers capture meaning

During pre-training these numbers are adjusted so that **similar words end up close together**:

```
"pizza"  ~ [ 0.8, 0.1,  0.6, ...]
"pasta"  ~ [ 0.7, 0.2,  0.5, ...]   <- close to pizza
"laptop" ~ [-0.3, 0.9, -0.4, ...]   <- far away
```

The classic example: **king − man + woman ≈ queen**. Directions in this space encode concepts like gender, plurality or "food-ness".

BERT adds three embeddings together

```
final input vector =  token embedding     (what word is this?)
                    + position embedding  (where is it? 1st, 2nd, 3rd...)
                    + segment embedding   (sentence A or B? from token_type_ids)
```

Position matters: "dog bites man" and "man bites dog" contain the same words in a different order. Modern LLMs use a cleverer position method called **RoPE** (rotary position embeddings), but the idea is the same.

Static vs contextual embeddings

The embedding *table* gives "bank" the same vector every time. After the transformer layers, "bank" in "river bank" and "bank" in "bank account" become **different** vectors. These are **contextual embeddings**, and they are what make transformers powerful.

> **Tiny detail**
> "Using BERT for sentence embeddings" usually means averaging the output vectors (**mean pooling**) or taking the [CLS] vector. Purpose-built `sentence-transformers` models do this much better than raw BERT, and are the usual choice for search and RAG.

11. The Transformer: how words look at each other

Inside the model are stacked identical layers: 12 in bert-base, 32–80+ in large LLMs. Each layer does two things.

(1) Self-attention

Every word asks: "Which other words in this sentence matter to *me*?"

> "The chef burned the toast because **it** was left too long." When processing "it", attention gives high weight to "toast" and low weight to "chef". The vector for "it" gets updated with toast information.

Mechanically, each token produces three vectors:

- **Query (Q)**: what I am looking for
- **Key (K)**: what I contain
- **Value (V)**: what I will share if you pick me

Each Query is compared with every Key to get scores; the scores become percentages (softmax); the percentages blend the Values. **Multi-head attention** does this 12 times in parallel so each head can focus on different relationships (grammar, meaning, references...). The attention_mask is applied right here, before softmax, so padding gets zero weight.

(2) Feed-forward network

Each token then passes on its own through a small neural network to "think about" what it just gathered. Stack 12 such layers, and each word's vector becomes rich and context-aware.

12. Model families: BERT vs GPT vs T5

	Encoder-only	Decoder-only	Encoder-Decoder
Examples	BERT, RoBERTa, DistilBERT, DeBERTa	GPT, Llama, Mistral, Qwen, Gemma	T5, BART
Attention	Sees both directions (whole sentence)	Sees only the left (past), never the future	Encoder sees all, decoder sees left
Trained by	Filling hidden words: "I love [MASK]"	Predicting the next word	Turning input text into output text
Good at	Understanding: classification, sentiment, name tagging, search	Generating: chat, writing, code	Transforming: translation, summarising
Analogy	A critic who reads the whole dish before judging	A chef cooking course after course, never seeing ahead	A translator who reads, then rewrites

What people call an "LLM" today is almost always **decoder-only**. BERT is still the best place to learn: it is small (110M parameters), trains fast on free hardware, and teaches every core concept.

bert-base facts	
Layers	12
Hidden size (vector length)	768
Attention heads	12
Parameters	~110 million
Max tokens	512

DistilBERT is about 40% smaller and 60% faster while keeping about 97% of BERT's performance, which makes it great for experiments.

13. AutoModel, AutoTokenizer and task heads

The "Auto" classes

Every architecture has its own class (`BertModel`, `LlamaModel`...). Instead of memorising them, Hugging Face's **Auto** classes read the model's `config.json` and pick the right class for you:

```
from transformers import AutoTokenizer, AutoModel
tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")
model     = AutoModel.from_pretrained("bert-base-uncased")
```

Swap the name for `"distilbert-base-uncased"` and the same code still works.

Body vs head

- `AutoModel` = the **body** only (embeddings + transformer layers). Output shape `[batch, tokens, 768]`. No decision.
- `AutoModelFor<Task>` = body + a **task head**.

Class	Head does	Example
`AutoModelForSequenceClassification`	One label for the whole text	Positive or negative review
`AutoModelForTokenClassification`	A label per token	Find names and places
`AutoModelForQuestionAnswering`	Find an answer span in a passage	"Where is the Lido?"
`AutoModelForMaskedLM`	Fill the blank	BERT's original task
`AutoModelForCausalLM`	Predict the next token	**All chat LLMs**
`AutoModelForSeq2SeqLM`	Text in, text out	T5 translation

What happens when you fine-tune BERT for classification

1. Load the pre-trained body. It already knows English.
2. Attach a brand-new head: a small layer mapping 768 numbers to your number of labels (e.g. 2).
3. Train on your examples. The new head learns from scratch; the body adjusts slightly.

Details that confuse every beginner

1. **The scary warning is normal.** "Some weights of BertForSequenceClassification were not initialized... You should probably TRAIN this model" just means your new head is random and needs fine-tuning.
2. **Outputs are logits, not probabilities.** Raw scores like `[-1.2, 3.4]`. Apply softmax to get `[1%, 99%]`.
3. **Pass `num_labels`** when loading a classifier; otherwise it assumes 2.
4. **`id2label` / `label2id`** store human names inside the model, so predictions show "positive" instead of `LABEL_0`.
5. **Three verbs**: `from_pretrained()` loads, `save_pretrained()` saves, `push_to_hub()` uploads. They work for models *and* tokenizers. Always save both together.
6. **`model.eval()` vs `model.train()`**: dropout (randomly switching off neurons during training) must be off when predicting. Forgetting `eval()` gives slightly random predictions.

14. Memory and precision: why big models are hard to train

A model is millions or billions of numbers called **weights** (parameters). Training means nudging them. Each number uses memory depending on its **precision**:

Precision	Bytes per number	Memory to load an 8B model
fp32 (full)	4	32 GB
fp16 / bf16 (half)	2	16 GB
int8	1	8 GB
4-bit	0.5	~4–5 GB

Training needs much more than loading. For every weight you also store its **gradient** (which direction to nudge it) and **optimizer states** (the popular Adam optimizer keeps two extra numbers per weight). The rule of thumb for full fine-tuning is **about 16 bytes per parameter**, so an 8B model needs well over **100 GB** of GPU memory. A free Colab GPU has about 15 GB. That gap is exactly why PEFT exists.

15. PEFT, LoRA and QLoRA

> **Cookbook analogy**
> The model is a **2,000-page professional cookbook**, and you want it to cook in your restaurant's style.
> • **Full fine-tuning** = rewriting every page. Expensive, slow, and you store a second 2,000-page book.
> • **LoRA** = keep the book untouched and add **sticky notes** on important pages ("more chilli here", "plate it this way"). The notes are tiny; book + notes cooks your way.
> • **QLoRA** = **shrink the book** to pocket size with a slightly lower-resolution photocopy, then add full-quality sticky notes. Fits in a small bag and still cooks almost as well.

PEFT: the umbrella term

PEFT = Parameter-Efficient Fine-Tuning: the family name for any method that trains only a small number of parameters and freezes the rest. Members:

- **LoRA**: the most popular by far
- **Prompt tuning / prefix tuning**: learn a few "virtual tokens" placed before your input
- **Adapters**: small extra layers inserted between existing layers
- **IA³**: learns tiny scaling factors

Hugging Face's `peft` library implements all of them. In practice most people use LoRA or QLoRA.

LoRA: how it actually works

History: published in 2021 by Edward Hu and colleagues at Microsoft. LoRA = **Low-Rank Adaptation**.

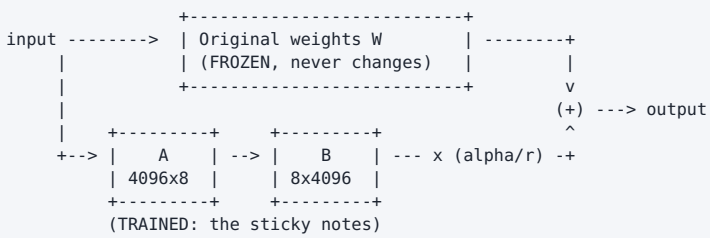
Key insight: when you fine-tune, the *change* to the weights is "simple". It does not need full detail and can be described with far fewer numbers.

```
A big weight grid in a Llama-size layer:  4096 x 4096  = 16,777,216 numbers
Full fine-tuning learns a change of the same size: 16.7 million numbers.

LoRA instead learns two THIN matrices and multiplies them:
   A:  4096 x 8      (tall and thin)
   B:     8 x 4096   (short and wide)
   A x B = 4096 x 4096   <- same shape as the full change

Numbers trained: (4096 x 8) + (8 x 4096) = 65,536
That is about 0.4% of 16.7 million.
```

The **8** is the **rank (r)**: think of it as how many sticky notes per page. Higher rank = more capacity, more memory. Common values: 8, 16, 32, 64.



Output = the original layer's answer **plus** a small LoRA correction.

Details most people do not know

1. **B starts at all zeros.** So at step 0 the correction is zero and the model behaves exactly like the original. Training grows the correction gently, which is why LoRA is stable.
2. `lora_alpha` is a volume knob: the correction is multiplied by `alpha / r`. A common habit is `alpha = 2 × r` (e.g. r=16, alpha=32).
3. `target_modules` chooses which layers get sticky notes. The original paper used only attention's query and value projections. Today, targeting **all linear layers** (q, k, v, o, gate, up, down) usually works better.
4. **No slowdown afterwards.** You can merge the notes into the book (`W_new = W + A×B`) with `merge_and_unload()`, giving a normal model with no extra computation.
5. **Adapters are tiny files**, often 10–100 MB for an 8B model. Keep one base model and swap adapters for different jobs.
6. **LoRA needs a higher learning rate** than full fine-tuning: around 1e-4 to 2e-4 instead of ~2e-5. Copying full fine-tuning settings often gives LoRA runs that barely learn.

QLoRA: LoRA on a compressed model

History: published in 2023 by Tim Dettmers and colleagues at the University of Washington (Dettmers also created `bitsandbytes`). Headline result: fine-tuning a **65-billion-parameter model on a single 48 GB GPU**.

What quantization means

Normally each weight is stored precisely (e.g. 16 bits). Quantization **rounds** each weight to one of a small set of allowed values. With 4 bits there are only **16 possible values**.

> Like reducing a photo from millions of colours to 16 colours: still surprisingly recognisable, and 4× smaller.

QLoRA's three tricks

1. **NF4 (4-bit NormalFloat).** Weights mostly cluster near zero in a bell-curve shape. NF4 places **more of its 16 allowed values near zero**, where most weights are, so less quality is lost than with evenly spaced 4-bit.
2. **Double quantization.** Quantization needs small "scale" numbers to undo the rounding. QLoRA quantizes those too, saving about 0.4 bits per parameter. Small, but it adds up over billions of weights.
3. **Paged optimizers.** When GPU memory spikes, optimizer data is moved temporarily to normal RAM instead of crashing with "CUDA out of memory".

```
Base model:   compressed to 4-bit, FROZEN       (~5 GB for an 8B model)
LoRA A and B: full precision (bf16), TRAINED    (tiny)
During each calculation, 4-bit weights are temporarily
"unpacked" to bf16, used, then discarded.
```

Method (8B model)	Approx. GPU memory to train
Full fine-tuning	100+ GB (several large GPUs)
LoRA (16-bit base)	~20–24 GB
QLoRA (4-bit base)	**~10–14 GB** (just about fits a free T4 GPU)

Trade-offs: QLoRA is a little slower than LoRA (all that unpacking) and very slightly lower quality. For learning and most real projects it is the best starting point.

16. The Python libraries in depth

Think of fine-tuning as running a restaurant kitchen. Each library has a role:

Library	Kitchen role	What it does
`torch` (PyTorch)	The stove and electricity	Maths engine made by Meta. Handles **tensors** (multi-dimensional arrays that can live on a GPU), automatic **gradients** (`loss.backward()` works out how to nudge every weight) and GPU control (`.to("cuda")`). Everything else sits on top of it.
`transformers`	The head chef	Hugging Face's core library: `AutoTokenizer` , `AutoModel*` , `pipeline()` (one-line inference), `Trainer` (full training loop with logging, evaluation, checkpoints), `TrainingArguments` (every training setting) and `generate()` (text generation). Supports hundreds of architectures.
`tokenizers`	The prep cook	The tokenizer engine written in **Rust**, far faster than pure Python. `transformers` uses it automatically ("Fast" tokenizers). You use it directly only to train a brand-new tokenizer.
`datasets`	The pantry	Loads data from the Hub (`load_dataset("imdb")`) or your CSV/JSON files. Built on **Apache Arrow** and memory-mapped, so a 50 GB dataset works on an 8 GB-RAM laptop by reading from disk on demand. `.map(fn, batched=True)` tokenizes every row fast and **caches** the result. Also `.train_test_split()` , `.filter()` , `.shuffle()` .
`accelerate`	The kitchen manager	Runs the same code on CPU, one GPU, many GPUs or several machines. Handles **mixed precision** (fp16/bf16). `device_map="auto"` spreads a big model across GPUs and CPU. `Trainer` uses it under the hood.
`peft`	The sticky notes	LoRA, QLoRA, prompt tuning and more. Key pieces: `LoraConfig` (r, alpha, target modules, dropout), `get_peft_model()` (attach adapters), `print_trainable_parameters()` (shows the small %), `merge_and_unload()` (merge notes into the model).
`trl`	The specialist chef for chat models	**Transformer Reinforcement Learning**. `SFTTrainer` (Supervised Fine-Tuning) is built for instruction/chat data: chat templates, packing short examples together, LoRA integration. Also `DPOTrainer` (teach preferences: "answer A beats B") and RLHF-style methods used to align assistant models.
`bitsandbytes`	The vacuum sealer	8-bit and 4-bit quantization (NF4) and memory-saving 8-bit optimizers. What makes QLoRA possible. Needs an NVIDIA GPU (CUDA). Used through `BitsAndBytesConfig` in `transformers` .
`evaluate`	The food critic	Ready-made metrics: accuracy, F1, precision/recall, BLEU, ROUGE. E.g. `evaluate.load("accuracy")` .
`huggingface_hub`	The delivery van	Login (`huggingface-cli login` / `notebook_login()`), downloads and uploads, `push_to_hub()` . Manages the cache folder `~/.cache/huggingface/` .
`safetensors`	Safe storage containers	File format for weights (`model.safetensors`). The old `pytorch_model.bin` used Python "pickle", which **can run hidden code** when loaded — a real risk with files from strangers. Safetensors stores only numbers: safe and faster. Now the default.
`sentence-transformers` (optional)	Embedding specialist	Produces good sentence embeddings for search and similarity; useful for RAG.
`wandb` / `tensorboard` (optional)	CCTV and logbook	Dashboards of training graphs: loss going down, accuracy going up.

How they connect in one training run

```
datasets ---------------------> loads your data
   |
tokenizers (via transformers) -> text becomes input_ids / attention_mask
   |
transformers -----------------> loads the model
   |
bitsandbytes -----------------> (QLoRA only) shrinks it to 4-bit
   |
peft -------------------------> attaches LoRA sticky notes
   |
trl / Trainer ----------------> runs the training loop
   |     +-- accelerate ------> manages GPU and precision
   |            +-- torch ----> does all the actual maths
   |
evaluate ---------------------> scores it
   |
huggingface_hub + safetensors -> saves and shares it
```

Install line (Google Colab or your own machine)

```
pip install -U transformers datasets accelerate peft trl bitsandbytes evaluate
```

On Colab, PyTorch is already installed. Choose **Runtime → Change runtime type → T4 GPU** for a free GPU.

17. Master cheat sheet

```
PIPELINE
  TEXT -> Tokenizer -> input_ids + attention_mask (+ token_type_ids for BERT)
       -> Embedding (ID -> 768 numbers, + position, + segment)
       -> Transformer layers (self-attention + feed-forward) x 12
       -> Head -> logits -> softmax -> answer

IDS AND VOCABULARY
  ID            = line number in the vocabulary file (vocab.txt / tokenizer.json)
  Vocabulary    = built once by a tokenizer algorithm, frozen with the model
  Add tokens    = tokenizer.add_tokens() + model.resize_token_embeddings()
  Tokenizer must ALWAYS match the model

BERT SPECIAL TOKENS
  [CLS] 101 = summary seat at the start (used for classification)
  [SEP] 102 = end / separator     [PAD] 0   = filler
  [UNK] 100 = unknown             [MASK] 103 = blank used in pre-training

PADDING AND MASKS
  Padding        = filler to make a batch rectangular; prefer dynamic padding
  Truncation     = cutting text that is too long (BERT max 512)
  attention_mask = 1 real / 0 padding   (the highlighter)
  token_type_ids = 0 sentence A / 1 sentence B   (BERT only)
  Decoder models: pad LEFT when generating; pad_token = eos_token

TOKENIZER ALGORITHMS
  BPE        merge most frequent pair        marker G-dot  GPT, Llama 3  never [UNK]
  WordPiece  merge most strongly linked      marker ##     BERT
  Unigram    start big, prune; probabilistic marker _      T5 (via SentencePiece)
  SentencePiece = a library (runs BPE or Unigram), treats spaces as characters
  1 token ~ 4 English characters ~ 3/4 word

MODELS
  Encoder (BERT) = understand | Decoder (GPT/Llama) = generate | Enc-Dec (T5) = transform
  AutoModel = body only        | AutoModelForX = body + task head
  Chat LLMs use AutoModelForCausalLM
  Outputs are logits -> softmax for probabilities
  "Weights not initialized" warning = normal (new head)
  model.eval() for predicting, model.train() for training

MEMORY
  Bytes per parameter: fp32=4  bf16=2  int8=1  4-bit=0.5
  Full fine-tuning ~ 16 bytes per parameter

FINE-TUNING METHODS
  Fine-tuning teaches behaviour/format/style; RAG gives knowledge
  PEFT  = family of "train only a little" methods
  LoRA  = frozen model + tiny A x B matrices (rank r); B starts at zero; mergeable
          typical: r=16, alpha=32, learning rate ~2e-4, target all linear layers
  QLoRA = LoRA + 4-bit NF4 base + double quantization + paged optimizers
```

18. Self-check questions

1. Where does the number 2293 for "love" come from, and does the number itself mean anything?
2. Why does BERT add [CLS] at the start, and which pre-training game taught it to be useful?
3. What would go wrong if you left out the attention_mask in a padded batch?
4. When is token_type_ids all zeros, and which models do not use it at all?
5. BPE vs WordPiece: one counts popularity, the other measures something else. What?
6. Why can byte-level BPE never produce [UNK]?
7. What is the difference between SentencePiece and Unigram?
8. Why is "bank" a different vector before and after the transformer layers?
9. You want a model to write replies in your company's tone. Encoder or decoder?
0. In LoRA, why does matrix B start at zero?
1. Name QLoRA's three tricks.
2. You want a model to answer questions about 500 internal PDFs. Fine-tuning or RAG first?

Answers in brief

1. Line number in vocab.txt; no meaning, just an address. 2. Its vector summarises the sentence; Next Sentence Prediction. 3. Real words would attend to padding and get polluted meaning, quietly. 4. With a single sentence; RoBERTa, DistilBERT, GPT, Llama. 5. How strongly two pieces belong together (pair count ÷ product of individual counts). 6. All 256 bytes are in its vocabulary. 7. SentencePiece is a library; Unigram is an algorithm it can run. 8. Embedding table is static; transformer layers add context. 9. Decoder. 10. So the model starts exactly like the original and training is stable. 11. NF4, double quantization, paged optimizers. 12. RAG first.

19. The small program that ties it all together

This program walks through every concept in this guide, in order, using real Hugging Face code. It runs on Google Colab (CPU is fine, GPU is faster) or any Python 3.10+ machine. It downloads bert-base-uncased (~440 MB) the first time.

```
pip install -U transformers peft torch
```

```
# finetune_concepts.py  -  every idea from this guide, in one small program
import torch
from transformers import AutoTokenizer, AutoModel, AutoModelForSequenceClassification
from peft import LoraConfig, get_peft_model

MODEL = "bert-base-uncased"
tok = AutoTokenizer.from_pretrained(MODEL)

# ---- Sections 4-5: tokens and IDs (IDs = line numbers in the vocabulary) ----
enc = tok("I love pizza")
print("1) input_ids      :", enc["input_ids"])
print("   tokens         :", tok.convert_ids_to_tokens(enc["input_ids"]))
print("   vocabulary size:", tok.vocab_size)               # 30522
print("   'love' is line :", tok.convert_tokens_to_ids("love"))

# ---- Section 9: WordPiece subwords and the ## marker ----
print("2) subwords       :", tok.tokenize("tokenization is unbelievably useful"))

# ---- Section 6: special tokens ----
for t in ["[PAD]", "[UNK]", "[CLS]", "[SEP]", "[MASK]"]:
    print(f"3) {t:7s} -> ID {tok.convert_tokens_to_ids(t)}")

# ---- Sections 7-8: padding + attention_mask ----
batch = tok(["I love pizza", "Yum"], padding=True, return_tensors="pt")
print("4) padded ids     :\n", batch["input_ids"])
print("   attention_mask :\n", batch["attention_mask"])

# ---- Section 8: token_type_ids with two sentences ----
pair = tok("Where is the Lido?", "The Lido is in Hyde Park.")
print("5) pair tokens    :", tok.convert_ids_to_tokens(pair["input_ids"]))
print("   token_type_ids :", pair["token_type_ids"])

# ---- Section 10: embeddings (static table) ----
body = AutoModel.from_pretrained(MODEL)
table = body.embeddings.word_embeddings.weight
print("6) embedding table shape:", tuple(table.shape))     # (30522, 768)

# ---- Sections 10-11: contextual embeddings - "bank" changes with context ----
def bank_vector(sentence):
    inputs = tok(sentence, return_tensors="pt")
    with torch.no_grad():
        out = body(**inputs).last_hidden_state[0]           # [tokens, 768]
    idx = inputs["input_ids"][0].tolist().index(tok.convert_tokens_to_ids("bank"))
    return out[idx]
v1 = bank_vector("we sat on the river bank")
v2 = bank_vector("i opened a bank account")
sim = torch.nn.functional.cosine_similarity(v1, v2, dim=0).item()
print(f"7) 'bank' similarity across contexts: {sim:.2f}  (1.00 would mean identical)")

# ---- Section 13: body + head, logits -> softmax ----
clf = AutoModelForSequenceClassification.from_pretrained(
    MODEL, num_labels=2,
    id2label={0: "negative", 1: "positive"}, label2id={"negative": 0, "positive": 1})
clf.eval()
with torch.no_grad():
    logits = clf(**tok("I love pizza", return_tensors="pt")).logits
print("8) logits (head is untrained, so random):", logits)
print("   softmax probabilities:", torch.softmax(logits, dim=-1))

# ---- Section 15: LoRA - freeze the book, add sticky notes ----
total = sum(p.numel() for p in clf.parameters())
cfg = LoraConfig(task_type="SEQ_CLS", r=8, lora_alpha=16, lora_dropout=0.1,
                 target_modules=["query", "value"])
lora_model = get_peft_model(clf, cfg)
print(f"9) total parameters: {total:,}")
lora_model.print_trainable_parameters()   # tiny % = LoRA matrices + new head

# ---- A tiny fine-tune: 4 examples, a few steps, watch the loss fall ----
texts  = ["I love this pizza", "Absolutely delicious", "This was awful", "Cold and tasteless"]
labels = torch.tensor([1, 1, 0, 0])
data = tok(texts, padding=True, return_tensors="pt")
opt = torch.optim.AdamW([p for p in lora_model.parameters() if p.requires_grad], lr=2e-4)

lora_model.train()                          # dropout on
for step in range(15):
```

```
    out = lora_model(**data, labels=labels)  # loss computed for us
    out.loss.backward()                      # gradients (torch)
    opt.step(); opt.zero_grad()              # nudge only the trainable weights
    if step % 5 == 0 or step == 14:
        print(f"10) step {step:2d}  loss {out.loss.item():.4f}")

lora_model.eval()                            # dropout off
with torch.no_grad():
    test = tok(["The pizza was wonderful", "Terrible service and food"],
               padding=True, return_tensors="pt")
    probs = torch.softmax(lora_model(**test).logits, dim=-1)
for sentence, p in zip(["The pizza was wonderful", "Terrible service and food"], probs):
    label = lora_model.config.id2label[int(p.argmax())]
    print(f"11) '{sentence}' -> {label} ({p.max():.0%})")

# ---- Merge sticky notes into the book, then save model + tokenizer together ----
merged = lora_model.merge_and_unload()
merged.save_pretrained("my-first-finetune")
tok.save_pretrained("my-first-finetune")
print("12) saved to ./my-first-finetune (model.safetensors + tokenizer files)")
```

What each printed line teaches

Line	What you will see	Concept (section)
1	`[101, 1045, 2293, ..., 102]` and the matching tokens	IDs are vocabulary line numbers; [CLS]/[SEP] added automatically (4, 5)
2	Pieces with `##`	WordPiece subwords (9)
3	0, 100, 101, 102, 103	Special tokens (6)
4	A rectangle with 0s, and a mask with matching 0s	Padding and attention_mask (7, 8)
5	0s for the question, 1s for the answer passage	token_type_ids (8)
6	`(30522, 768)`	Embedding table: one row per ID (10)
7	A similarity well below 1.00	Contextual embeddings (10, 11)
8	Random logits and ~50/50 probabilities, plus the "not initialized" warning	Body + new head; logits vs softmax (13)
9	~110 million total, well under 1% trainable	LoRA (15)
10	Loss getting smaller	Training with gradients (14, 16)
11	Predictions with human labels	id2label, eval mode (13)
12	Saved folder	Merging adapters; save model and tokenizer together (13, 15)

Be honest about the result

Four training examples are only enough to *see* the mechanics. The predictions in line 11 may be right or wrong; real fine-tuning uses hundreds to thousands of examples, a validation set, and the `Trainer` or `SFTTrainer` classes.

Experiments to try next

- Change `MODEL` to `"distilbert-base-uncased"`. Note that token_type_ids disappear from line 5, and the LoRA target modules become `["q_lin", "v_lin"]`.
- Change `r=8` to `r=32` and watch the trainable percentage grow.
- Tokenize a sentence in Gujarati or Hindi and count the tokens compared with English.
- Remove `attention_mask` from the batch and see the predictions shift.